41. mTLS and Service Identity Intro

41. mTLS and Service Identity Intro

Phase: Phase 5 — HTTP, TLS, and APIs
Estimated time: 1–2 days
Slug: `41-tls-mtls`

Learning objectives

- Explain client certificates
- Contrast mTLS vs token auth
- Inspect cert fields for clients
- Know mesh (Istio/Linkerd) uses mTLS

Prerequisites

Complete prior lessons. Prefer a Linux VM for `ip`/`ss`/`nft` examples; macOS works for many `curl`/`dig` drills. Type every command yourself and keep a lab notes file.

Why this matters

Zero-trust service mesh wants identity, not IPs. mTLS client certs prove workload identity — different failure mode than server-only TLS.

Core concepts

What you must be able to explain:

- Explain client certificates
- Contrast mTLS vs token auth
- Inspect cert fields for clients
- Know mesh (Istio/Linkerd) uses mTLS

Mental model for `41-tls-mtls` (exercise kind `mtls_quiz`):

- Prefer **measurable** checks (exit codes, ICMP replies, DNS rcode, HTTP status, socket state) over vibes.
- Separate **reachability** (can packets flow?) from **correctness** (right service, cert, route, identity).
- Ask: *which layer's tool falsifies my current hypothesis?* Then run that tool.
- Record the exact command that proved the root cause — that becomes your playbook snippet.

Worked commands

Run these on a lab host. Linux VM preferred for `ip`/`ss`/`tcpdump`/`nft`; macOS is fine for many `curl`/`dig`/`ping` drills. Your addresses and timings will differ — match the *shape* of the output and the conclusion you draw.

Example 1

```
curl -I https://example.com 2>&1 | head -15
```

Sample output:

```
HTTP/2 200
content-type: text/html
cache-control: max-age=...
```

Interpretation: Response line and headers are L7 evidence after TCP/TLS succeeded.

Example 2

```
curl -sS -o /dev/null -w 'code=%{http_code} ttfb=%{time_starttransfer}
total=%{time_total} ver=%{http_version}\n' https://example.com
```

Sample output:

```
code=200 ttfb=0.120 total=0.180 ver=2
```

Interpretation: Write-out metrics separate connect delay from first-byte and total time — essential for gateway/CDN debates.

Example 3

```
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/
null | openssl x509 -noout -subject -dates
```

Sample output:

```
subject=CN = example.com
notBefore=...
notAfter=...
```

Interpretation: Certificate identity and expiry are independent of HTTP 200. SNI (`-servername`) selects the cert on shared IPs.

Mini-lab

1. Re-type every worked command for **mTLS and Service Identity Intro** and paste outputs into lab notes.
2. Introduce one controlled failure in the lab (wrong DNS, firewall drop, bad route) and recover using only this lesson's tools.
3. Write a 5-line playbook: symptom → first command → expected healthy output.
4. Complete the exercise stubs; then explain each function's return value without looking.

Failure modes

Symptom	Likely cause	Next check
Long-lived client certs without rotat…	See common mistakes	Re-run lesson commands
Skipping revocation strategy	See common mistakes	Re-run lesson commands
Mixing mTLS and permissive plaintext …	See common mistakes	Re-run lesson commands
Timeout with no RST	Silent drop / blackhole	tcpdump + compare path

Study tip for mTLS and Service Identity Intro

Before each new command, write a one-line hypothesis. If the result falsifies it, update the hypothesis instead of stacking random tools. Note equivalents across OS (`ip` ↔ `ifconfig`/`route`, `ss` ↔ `netstat`). Re-run two commands from this lesson tomorrow from memory and explain the sample outputs out loud.

Evidence habit

When pages are down, prefer evidence (`ip route get`, resolver answer, packet capture, listener list) over intuition. Document **command → output → conclusion** in a lab notebook so the same path is reusable on-call. Change one variable at a time; write a one-line hypothesis before each new tool.

Common mistakes

- Long-lived client certs without rotation
- Skipping revocation strategy
- Mixing mTLS and permissive plaintext elsewhere

Exercises

Work in `exercises/41-tls-mtls/`. Read that folder's `README.md` before coding.

```
cd exercises/41-tls-mtls
chmod +x test.sh exercise.sh 2>/dev/null || true
./test.sh
```

What you will build

Edit `exercise.sh` and implement `mtls_means`, `mtls_both_present`, `service_identity_bound_to`. Each function is a tiny retrieval drill for **mTLS and Service Identity Intro** — return the exact tokens/flags/labels from the lesson. Keep the comment

above each function describing when you would use that fact on-call. Run `./test.sh` until all 3 assertions pass.

Study loop

- **A (Drill):** make tests pass
- **B (Apply):** run 2–3 real commands from this lesson in your lab and note outputs
- **C (Challenge):** rewrite answers from memory next day

Do not open `solutions/41-tls-mtls/` until after an honest attempt.

Self-check

Answer without scrolling up:

1. mTLS means?
2. curl flags for client cert?
3. Mesh relation?

Further reading

- https://www.rfc-editor.org/rfc/rfc8446